

DP-800 Laboratorio 02 - Implement programmability objects with SQL

Índice

1. Restauración de la base de datos.....	1
2. Verificar la conexión con la base de datos.....	3
3. Creación de una vista para simplificar las consultas	4
4. Crear un procedimiento almacenado para procesar un pedido	5
5. Crear una función escalar para cálculos reutilizables	7
6. Crear una función con valor de tabla en líneas (TVF)	9
7. Crear un desencadenador (trigger) para registrar cambios	10

1. Restauración de la base de datos

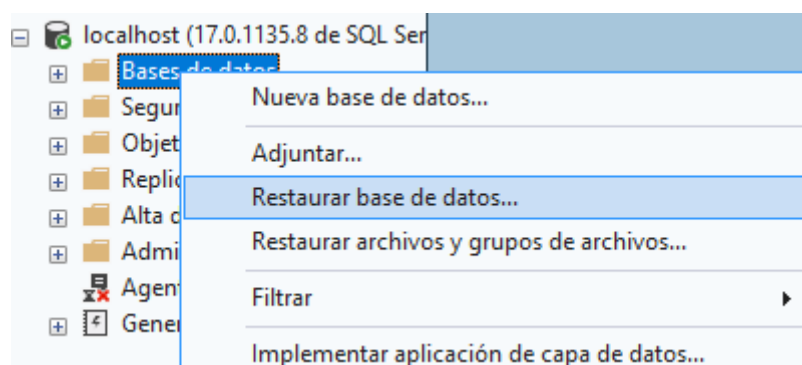
Accedemos a esta página web <https://learn.microsoft.com/en-us/sql/samples/adventureworks-install-configure?view=sql-server-ver17&tabs=ssms> .

Bajando un poco veremos una tabla con ficheros con nombre AdventureWorks2025.bak

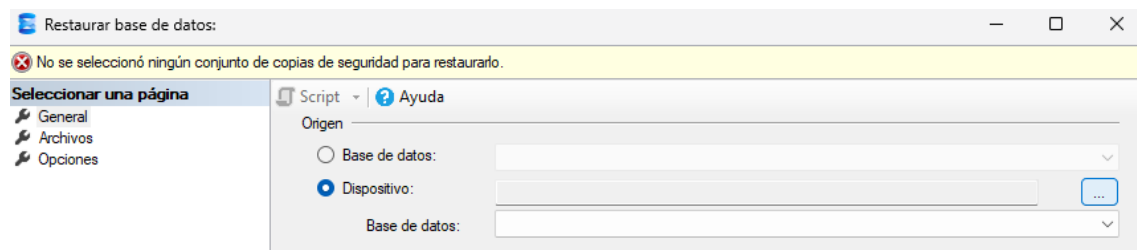
OLTP	Data Warehouse	Lightweight
AdventureWorks2025.bak	AdventureWorksDW2025.bak	AdventureWorksLT2025.bak
AdventureWorks2022.bak	AdventureWorksDW2022.bak	AdventureWorksLT2022.bak
AdventureWorks2019.bak	AdventureWorksDW2019.bak	AdventureWorksLT2019.bak
AdventureWorks2017.bak	AdventureWorksDW2017.bak	AdventureWorksLT2017.bak
AdventureWorks2016.bak	AdventureWorksDW2016.bak	AdventureWorksLT2016.bak
AdventureWorks2016_EXT.bak	AdventureWorksDW2016_EXT.bak	N/A
AdventureWorks2014.bak	AdventureWorksDW2014.bak	AdventureWorksLT2014.bak
AdventureWorks2012.bak	AdventureWorksDW2012.bak	AdventureWorksLT2012.bak
AdventureWorks2008R2.bak	AdventureWorksDW2008R2.bak	N/A

Nos descargamos el primero fichero de la columna Lightweight.

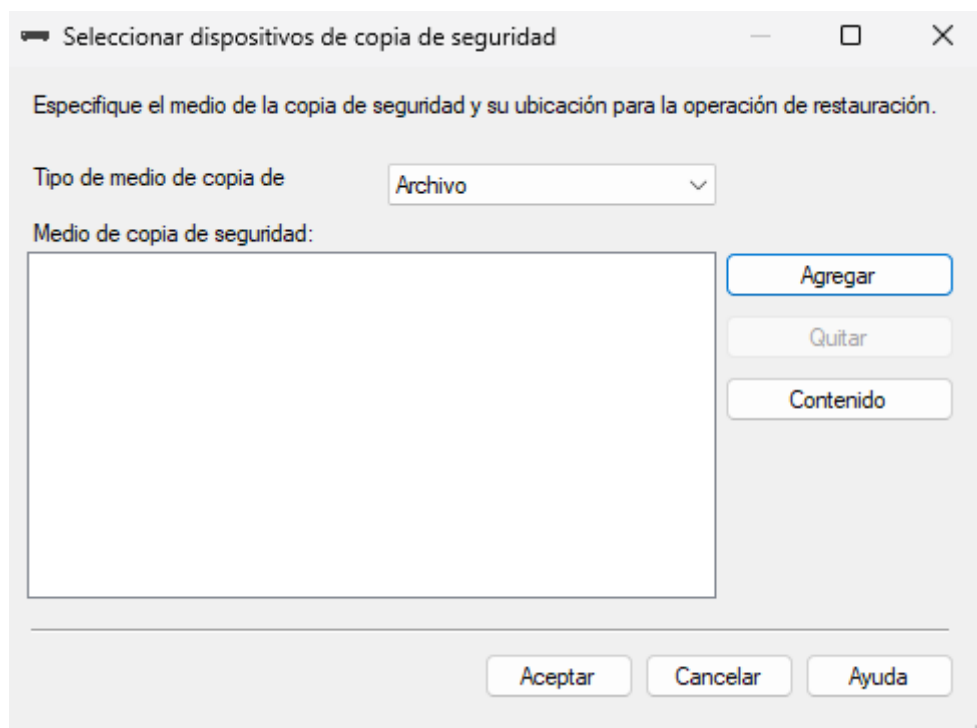
Para restaurarla abrimos nuestro SSMS, hacemos click derecho en ‘Bases de datos’ y pulsamos en ‘Restaurar base de datos’.



Se abrirá una ventana en la que debemos marcar 'Dispositivo' y pulsar los tres puntitos.



Se abrirá otra ventana en la que debemos pulsar 'Agregar' y seleccionar el fichero que acabamos de descargar.



Pulsamos en 'Aceptar' y se nos completarán todos los campos automáticamente por lo que solo queda pulsar de nuevo en 'Aceptar'.

2. Verificar la conexión con la base de datos

Para verificar que se ha restaurado correctamente el backup anterior, ejecutamos las siguientes consultas:

```
-- Verify key tables in AdventureWorksLT
SELECT TOP (5) CustomerID, FirstName, LastName
FROM SalesLT.Customer;

SELECT TOP (5) SalesOrderID, OrderDate, CustomerID
FROM SalesLT.SalesOrderHeader;

SELECT TOP (5) ProductID, Name, ListPrice
FROM SalesLT.Product;
```

Resultados		Mensajes	
	CustomerID	FirstName	LastName
1	1	Orlando	Gee
2	2	Keith	Harris
3	3	Donna	Carreras
4	4	Janet	Gates
5	5	Lucy	Harington

	SalesOrderID	OrderDate	CustomerID
1	71774	2008-06-01 00:00:00.000	29847
2	71776	2008-06-01 00:00:00.000	30072
3	71780	2008-06-01 00:00:00.000	30113
4	71782	2008-06-01 00:00:00.000	29485
5	71783	2008-06-01 00:00:00.000	29957

	ProductID	Name	ListPrice
1	680	HL Road Frame - Black, 58	1431,50
2	706	HL Road Frame - Red, 58	1431,50
3	707	Sport-100 Helmet, Red	34,99
4	708	Sport-100 Helmet, Black	34,99
5	709	Mountain Bike Socks, M	9,50

3. Creación de una vista para simplificar las consultas

Ejecutamos el siguiente código para facilitar las consultas del resto de apartados:

```
CREATE OR ALTER VIEW SalesLT.vCustomerOrders AS
SELECT
    c.CustomerID,
    CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
    h.SalesOrderID,
    h.OrderDate
FROM SalesLT.Customer c
INNER JOIN SalesLT.SalesOrderHeader h ON c.CustomerID = h.CustomerID;
```

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-21T11:41:47.0016878+02:00

Realizamos la siguiente consulta para comprobar que se ha creado correctamente:

```
SELECT TOP (5) *
FROM SalesLT.vCustomerOrders
ORDER BY OrderDate DESC;
```

	CustomerID	CustomerName	SalesOrderID	OrderDate
1	29847	David Hodgson	71774	2008-06-01 00:00:00.000
2	30072	Andrea Thomsen	71776	2008-06-01 00:00:00.000
3	30113	Raja Venugopal	71780	2008-06-01 00:00:00.000
4	29485	Catherine Abel	71782	2008-06-01 00:00:00.000
5	29957	Kevin Liu	71783	2008-06-01 00:00:00.000

4. Crear un procedimiento almacenado para procesar un pedido

Creamos el siguiente procedimiento para que se cree un pedido automáticamente (inserte los datos en las tablas correspondientes):

```
CREATE OR ALTER PROCEDURE dbo.AddOrderLineItem
    @SalesOrderID INT,
    @ProductID INT,
    @Quantity INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;

    -- Use Product ListPrice as UnitPrice
    DECLARE @UnitPrice DECIMAL(18,2);
    SELECT @UnitPrice = CAST(ListPrice AS DECIMAL(18,2))
    FROM SalesLT.Product
    WHERE ProductID = @ProductID;

    IF @UnitPrice IS NULL
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50010, 'Invalid ProductID specified.', 1;
    END

    -- Ensure SalesOrderID exists
    IF NOT EXISTS (SELECT 1 FROM SalesLT.SalesOrderHeader WHERE SalesOrderID = @SalesOrderID)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50011, 'Invalid SalesOrderID specified.', 1;
    END

    -- Insert line item (no discount)
    INSERT INTO SalesLT.SalesOrderDetail (SalesOrderID, OrderQty, ProductID, UnitPrice, UnitPriceDiscount)
    VALUES (@SalesOrderID, @Quantity, @ProductID, @UnitPrice, 0);

    -- Update header subtotal based on current line totals
    UPDATE h
    SET SubTotal = d.SumLineTotal,
        ModifiedDate = SYSUTCDATETIME()
    FROM SalesLT.SalesOrderHeader h
    INNER JOIN (
        SELECT SalesOrderID, SUM(LineTotal) AS SumLineTotal
        FROM SalesLT.SalesOrderDetail
        WHERE SalesOrderID = @SalesOrderID
        GROUP BY SalesOrderID
    ) d ON d.SalesOrderID = h.SalesOrderID;

    COMMIT TRANSACTION;
END;
```



Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-21T11:43:06.3272973+02:00

Llamamos al procedimiento para comprobar que realmente funciona y luego realizamos dos consultas para ver si se ha insertado en las tablas:

```
-- Add a line item to an existing order (choose a valid SalesOrderID)
DECLARE @SalesOrderID INT = (SELECT TOP 1 SalesOrderID
                              FROM SalesLT.SalesOrderHeader
                              ORDER BY SalesOrderID DESC);
EXEC dbo.AddOrderLineItem @SalesOrderID = @SalesOrderID,
                          @ProductID = 680,
                          @Quantity = 1; -- adjust ProductID as needed

SELECT TOP (5) *
FROM SalesLT.SalesOrderDetail
WHERE SalesOrderID = @SalesOrderID
ORDER BY SalesOrderDetailID DESC;

SELECT SalesOrderID, SubTotal, TaxAmt, Freight, TotalDue
FROM SalesLT.SalesOrderHeader
WHERE SalesOrderID = @SalesOrderID;
```

Resultados

Mensajes

	SalesOrderID	SalesOrderDetailID	OrderQty	ProductID	UnitPrice	UnitPriceDiscount	LineTotal	rowguid	ModifiedDate
1	71946	113407	1	680	1431.50	0.00	1431.500000	6B3F33D7-8115-4327-BB36-0FB93F1CE2E8	2026-09-21 11:43:32.927
2	71946	113406	1	916	31.584	0.00	31.584000	B571CEE0-A90A-4E76-9796-379B55352A28	2008-06-01 00:00:00.000

	SalesOrderID	SubTotal	TaxAmt	Freight	TotalDue
1	71946	1463.084	3.1163	0.9738	1467.1741

5. Crear una función escalar para cálculos reutilizables

Creemos una función que devuelva el Order Total:

```
CREATE OR ALTER FUNCTION dbo.fnOrderTotal (@OrderID INT)
RETURNS DECIMAL(18,2)
AS
BEGIN
    DECLARE @Total DECIMAL(18,2);

    SELECT @Total = SUM(LineTotal)
    FROM SalesLT.SalesOrderDetail
    WHERE SalesOrderID = @OrderID;

    RETURN ISNULL(@Total, 0.00);
END;
```

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-21T11:44:08.9121356+02:00

Comprobamos con la siguiente consulta:

```
SELECT d.SalesOrderID, dbo.fnOrderTotal(d.SalesOrderID) AS OrderTotal
FROM SalesLT.SalesOrderDetail d
GROUP BY d.SalesOrderID
ORDER BY d.SalesOrderID DESC;
```

	SalesOrderID	OrderTotal
1	71946	1463.08
2	71938	74160.23
3	71936	79589.62
4	71935	5533.87
5	71923	96.11
6	71920	2527.13
7	71917	37.76
8	71915	1732.89
9	71902	59894.21
10	71899	1856.21
11	71898	53248.69
12	71897	10585.05
13	71895	221.26
14	71885	524.66
15	71867	858.90
16	71863	2777.14
17	71858	11528.84
18	71856	500.30

Consulta e... | localhost (17.0 RTM) | sa (80)

6. Crear una función con valor de tabla en líneas (TVF)

Creamos una función que nos devuelve una tabla a partir de un SELECT:

```
CREATE OR ALTER FUNCTION dbo.GetCustomerOrders (@CustomerID INT)
RETURNS TABLE
AS
RETURN
(
    SELECT
        h.SalesOrderID,
        h.OrderDate
    FROM SalesLT.SalesOrderHeader h
    WHERE h.CustomerID = @CustomerID
);
```

Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-21T11:45:05.4062452+02:00

Realizamos la siguiente consulta para el correcto funcionamiento de la función creada:

```
SELECT *
FROM dbo.GetCustomerOrders(29929)
ORDER BY OrderDate DESC;
```

Resultados		Mensajes	
	SalesOrderID	OrderDate	
1	71902	2008-06-01 00:00:00.000	

Otra comprobación más:

```
SELECT CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName, o.SalesOrderID, o.OrderDate
FROM SalesLT.Customer c
CROSS APPLY dbo.GetCustomerOrders(c.CustomerID) o
WHERE c.CustomerID = 29929;
```

Resultados		Mensajes	
	CustomerName	SalesOrderID	OrderDate
1	Jeffrey Kurtz	71902	2008-06-01 00:00:00.000

7. Crear un desencadenador (trigger) para registrar cambios

Creamos un trigger que al hacer un insert o update en la tabla SalesOrderDetail haga un insert en la tabla OrderAudit:

```
-- Audit table
IF OBJECT_ID('dbo.OrderAudit') IS NULL
BEGIN
    CREATE TABLE dbo.OrderAudit (
        AuditID      INT IDENTITY(1,1) PRIMARY KEY,
        OrderID      INT NOT NULL,
        OldTotal     DECIMAL(18,2) NULL,
        NewTotal     DECIMAL(18,2) NULL,
        ChangedAt    DATETIME2 NOT NULL DEFAULT SYSUTCOATETIME()
    );
END
GO

-- Trigger on order details updates
CREATE OR ALTER TRIGGER SalesLT.trg_LogOrderTotalChange
ON SalesLT.SalesOrderDetail
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    ;WITH AffectedOrders AS (
        SELECT SalesOrderID FROM inserted
        UNION
        SELECT SalesOrderID FROM deleted
    ),
    -- New totals from the base table (already reflects changes)
    NewTotals AS (
        SELECT d.SalesOrderID, SUM(d.OrderQty * d.UnitPrice) AS Total
        FROM SalesLT.SalesOrderDetail d
        INNER JOIN AffectedOrders a ON d.SalesOrderID = a.SalesOrderID
        GROUP BY d.SalesOrderID
    ),
    -- Contribution of the newly inserted/updated rows
    InsertedTotals AS (
        SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
        FROM inserted
        GROUP BY SalesOrderID
    ),
    -- Contribution of the previous row versions (empty on INSERT)
    DeletedTotals AS (
        SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
        FROM deleted
        GROUP BY SalesOrderID
    )
    INSERT INTO dbo.OrderAudit (OrderID, OldTotal, NewTotal)
    SELECT
        n.SalesOrderID,
        n.Total - ISNULL(i.Total, 0) + ISNULL(d.Total, 0) AS OldTotal,
        n.Total AS NewTotal
    FROM NewTotals n
    LEFT JOIN InsertedTotals i ON n.SalesOrderID = i.SalesOrderID
    LEFT JOIN DeletedTotals d ON n.SalesOrderID = d.SalesOrderID;
END;
```



Mensajes

Los comandos se han completado correctamente.

Hora de finalización: 2026-09-21T11:46:07.2787815+02:00

Comprobamos el funcionamiento del trigger con el siguiente UPDATE y SELECT:

```
-- Update an order detail to change the total
UPDATE d
SET OrderQty = OrderQty + 1
FROM SalesLT.SalesOrderDetail d
WHERE d.SalesOrderID = (SELECT TOP 1 SalesOrderID FROM SalesLT.SalesOrderHeader ORDER BY SalesOrderID DESC);

SELECT TOP (5) *
FROM dbo.OrderAudit
ORDER BY AuditID DESC;
```

Resultados		Mensajes			
	AuditID	OrderID	OldTotal	NewTotal	ChangedAt
1	1	71946	1463.08	2926.17	2026-09-21 09:46:33.6724367